

Lauri Jokinen

SOLVING A RUBIK'S CUBE WITH COMPUTER VISION

Bachelor's Thesis
Faculty of Engineering and Natural Sciences
Examiner: Roel Pieters
January 2024

ABSTRACT

Lauri Jokinen: Solving a Rubik's Cube with Computer Vision
Bachelor's Thesis
Tampere University
Bachelor of Science (Technology), Automation Engineering
January 2024

In this thesis, computer vision tools for solving the Rubik's Cube are examined and developed. The main goal of the thesis is to develop a program that can detect the state of a scrambled Rubik's Cube via webcam, compute a solution for it, and guide human user to execute the solution. This is done in Python using software libraries OpenCV for computer vision tasks and Kociemba for solution computing. Besides creating the program, the goal of this thesis is to examine commonly used tools and practices in the field of computer vision, and to present its applications on other fields of activity.

In the first part of the thesis, the relevant background information is presented for the reader. Structure and mechanics of the Rubik's Cube are discussed, existing solving algorithms are compared to each other, uses and evolution of computer vision are examined, and three already existing remarkable robotic Rubik's Cube solvers are presented.

In the second part of the thesis, the implementation of the program is discussed. The program is implemented using traditional computer vision techniques that are based on detecting different colors and shapes from an image following hard-coded rules rather than utilizing machine learning, although advanced techniques that take use of machine learning are briefly discussed in the background info section. To compute solutions for given states of the cube, an algorithm developed by German mathematician Herbert Kociemba is used. The algorithm allows the program to find near-optimal solutions in minimal time.

The programming process was demanding, and it took lots of debugging and error fixing to get the program to work in desired way. Biggest challenges turned out to be the changes in lighting conditions of the environment, and some specific patterns of the cube pieces that often led to flawed conclusions of the state of the cube. However, the final version of the program meets the goal of solving the Rubik's Cube from any starting state and does not seem to be prone to errors when used properly.

A useful improvement for the program would be an automatic calibration of the limit values of the colors to get them to match the prevailing lighting conditions. Also, the detection features could be improved in a way that it would not be so strict in which order the user shows each face to the camera and how the cube is held during the detection and solving. The program could also be used in future applications where the human user would be replaced by robotic hardware handling the cube, making the system completely autonomous.

Keywords: Computer vision, object detection, Rubik's Cube

The originality of this thesis has been checked using the Turnitin OriginalityCheck service.

TIIVISTELMÄ

Lauri Jokinen: Rubikin kuution ratkaiseminen konenäön avulla
Kandidaatintyö
Tampereen yliopisto
Teknisten tieteiden kandidaattiohjelma, Automaatiotekniikka
Tammikuu 2024

Tässä työssä tutkitaan ja kehitetään konenäön työkaluja Rubikin kuution ratkaisemiseksi. Työn ensisijainen tavoite on luoda tietokoneohjelma, joka tunnistaa sekoitetun Rubikin kuution alkutilan verkkokameran välityksellä, etsii kyseiselle alkutilalle ratkaisun ja ohjeistaa ohjelman käyttäjää toteuttamaan tämän ratkaisun. Ohjelma toteutetaan Python-ohjelmointikielellä käyttäen hyväksi ohjelmistokirjastoja OpenCV ja Kociemba, joista ensin mainittua käytetään konenäön tehtävissä ja jälkimmäisenä mainittua kuution alkutilaan sopivan ratkaisun etsimisessä. Ohjelman toteuttamisen lisäksi työn tavoitteena on tutustua konenäön tehtävien parissa yleisesti käytettäviin työkaluihin ja toimintamalleihin sekä esitellä konenäön käyttökohteita eri toimialoilla.

Työn ensimmäisessä osassa lukijalle esitetään oleellista taustatietoa työn aiheeseen liittyen. Ensin käsitellään Rubikin kuution rakennetta ja mekaniikkaa, minkä jälkeen vertaillaan olemassa olevia ratkaisualgoritmeja keskenään. Sen jälkeen käydään läpi konenäön käyttökohteita ja sen historiallista kehitystä, ja lopuksi esitellään kolme kuuluisaa Rubikin kuution ratkaisemiseen suunniteltua robottia.

Työn toisessa vaiheessa käsitellään ohjelman toteutusta. Ohjelmassa hyödynnetään manuaalisesti koodattuja, perinteisiä, muotojen ja värien tunnistamiseen pohjautuvia konenäön tekniikoita kehittyneiden koneoppimismallien sijaan, joskin myös kehittyneempiä koneoppimista hyödyntäviä tekniikoita käsitellään pintapuolisesti työn ensimmäisessä osassa osana oleellisia taustatietoja. Kuution alkutilaa vastaavan ratkaisun etsimiseen käytetään saksalaisen matemaatikon Herbert Kociemban kehittämää algoritmia, joka mahdollistaa lähes optimaalisten ratkaisujen löytämisen minimaalisessa ajassa.

Ohjelmointiprosessi oli vaativa ja haluttujen toiminnallisuuksien saavuttaminen vaati runsaasti iteratiivista virheiden etsintää ja korjausta. Muuttuva valaistus ja tietyt hankalasti tunnistettavat kuution alkutilat osoittautuivat suurimmiksi haasteiksi. Vaikeuksista huolimatta, ohjelman lopullinen versio täyttää sille asetetut tavoitteet: kuutio saadaan ratkaistua mistä tahansa alkutilasta eikä ohjelma vaikuta olevan altis virhetilanteille, kun käyttäjä seuraa annettuja ohjeita.

Ohjelmaa voisi edelleen kehittää lisäämällä siihen automaattisen värien raja-arvojen kalibroinnin, joka sovittaisi raja-arvot vallitsevaan valaistukseen sopiviksi. Myös kuution tilan tunnistamista voitaisiin kehittää joustavampaan suuntaan siten, että käyttäjä voisi näyttää kuution sivut kameralle haluamassaan järjestyksessä ja pitää kuutiota missä tahansa asennossa. Tulevaisuudessa ohjelmaa voitaisiin hyödyntää myös osana täysin autonomista robottia, jossa ihmiskäyttäjä olisi korvattu mekaanisilla osilla, jotka kääntelevät kuutiota ja sen sivuja ohjelman käskyjen perusteella.

Avainsanat: Konenäkö, hahmontunnistus, Rubikin kuutio

Tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck –ohjelmalla.

CONTENTS

1. INTRODUCTION	1
2. BACKGROUND FOR CUBE DETECTION AND SOLUTION COMPUTING	2
2.1 The Rubik's Cube	2
2.1.1 The structure and mechanics of the cube	2
2.1.2 Computing solutions	4
2.2 Computer vision as a crucial part of robotics	4
2.2.1 Uses	5
2.2.2 Evolution of object detection	6
2.2.3 Tools and platforms	7
2.3 Existing robotic solutions for the Rubik's Cube	8
2.3.1 Guinness World Record robot from MIT	8
2.3.2 One-handed cube solver from OpenAI	8
2.3.3 Gan Robot	9
3. IMPLEMENTATION	11
3.1 Interaction with the user	11
3.2 The model of the cube	13
3.3 Detection	15
3.3.1 Distinguishing the cube from the background	15
3.3.2 Locating the center points	20
3.3.3 Detecting the colors of individual stickers	26
3.4 Solving the cube	28
4. RESULTS AND POTENTIAL IMPROVEMENTS	32
5. CONCLUSIONS	33
REFERENCES	34

LIST OF FIGURES AND PROGRAMS

<i>Figure 1. The terminology of the cube pieces.</i>	<i>3</i>
<i>Figure 2. The representation of the state of the cube.</i>	<i>3</i>
<i>Figure 3. Number of publications in object detection from 1998 to 2021 [9].</i>	<i>6</i>
<i>Figure 4. Distracting OpenAI's Rubik's Cube solver with stuffed animal [3].</i>	<i>9</i>
<i>Figure 5. Gan Robot [4].</i>	<i>10</i>
<i>Figure 6. Guiding the user to rotate the cube counterclockwise.</i>	<i>12</i>
<i>Figure 7. Asking the user if the cube was detected correctly.</i>	<i>12</i>
<i>Figure 8. Guiding the user to turn the left side of the cube.</i>	<i>13</i>
<i>Figure 9. Filtering out dark colors.</i>	<i>16</i>
<i>Figure 10. Minimum and maximum distances from the camera. Top left: too close, top right: at the minimum distance, bottom left: maximum distance, bottom right: too far.</i>	<i>17</i>
<i>Figure 11. Maximal rotations allowed. Top left: too much counterclockwise rotation, top right: maximal counterclockwise rotation, bottom left: maximum clockwise rotation, bottom right: too much clockwise rotation.</i>	<i>18</i>
<i>Figure 12. The exact contour of the detected cube.</i>	<i>19</i>
<i>Figure 13. The smallest rectangle that can be drawn around the detected cube.</i>	<i>19</i>
<i>Figure 14. Width w, height h, and rotational angle θ when $0^\circ < \theta \leq 30^\circ$.</i>	<i>20</i>
<i>Figure 15. Width w, height h, and rotational angle θ when $60^\circ \leq \theta \leq 90^\circ$.</i>	<i>20</i>
<i>Figure 16. Step sizes during iteration.</i>	<i>22</i>
<i>Figure 17. Outlining all the blue spots on the image.</i>	<i>27</i>
<i>Figure 18. Ring of green stickers surrounding a red center.</i>	<i>28</i>
<i>Figure 19. The sticker positions using Singmaster notation [5].</i>	<i>29</i>
<i>Figure 20. Achieving the goal.</i>	<i>31</i>
Program 1. <i>The data structure of the state of the cube.</i>	<i>14</i>
Program 2. <i>Calculating the locations of sticker centers (referred to as "piece centers" in the code).</i>	<i>25</i>

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
CNN	Convolutional Neural Networks
ADR	Automatic Domain Randomization
IDE	Integrated Development Environment
RGB	Red-Green-Blue
HSV	Hue-Saturation-Value
<i>Wh</i>	white centered side of the cube
<i>Or</i>	orange centered side of the cube
<i>Bl</i>	blue centered side of the cube
<i>Re</i>	red centered side of the cube
<i>Gr</i>	green centered side of the cube
<i>Ye</i>	yellow centered side of the cube
<i>Wh1...Wh9</i>	stickers of the side <i>Wh</i> proceeding in reading direction
<i>Or1...Or9</i>	stickers of the side <i>Or</i> proceeding in reading direction
<i>Bl1...Bl9</i>	stickers of the side <i>Bl</i> proceeding in reading direction
<i>Re1...Re9</i>	stickers of the side <i>Re</i> proceeding in reading direction
<i>Gr1...Gr9</i>	stickers of the side <i>Gr</i> proceeding in reading direction
<i>Ye1...Ye9</i>	stickers of the side <i>Ye</i> proceeding in reading direction
<i>w</i>	width of the cube
<i>h</i>	height of the cube
θ	rotational angle of the cube
x_w	x-component of width <i>w</i>
y_w	y-component of width <i>w</i>
x_h	x-component of height <i>h</i>
y_h	y-component of height <i>h</i>
x_0	x-coordinate of the top left corner of the cube
y_0	y-coordinate of the top left corner of the cube
w_{step}	width-directional step size
h_{step}	height-directional step size
dx_w	x-directional transition per w_{step}
dy_w	y-directional transition per w_{step}
dx_h	x-directional transition per h_{step}
dy_h	y-directional transition per h_{step}
dx	x-directional total transition
dy	y-directional total transition
<i>n</i>	number of steps in width-direction
<i>m</i>	number of steps in height-direction
<i>i</i>	row number of a sticker
<i>j</i>	column number of a sticker
$x_{sticker}$	x-coordinate of the center of a sticker
$y_{sticker}$	y-coordinate of the center of a sticker
<i>U</i>	side of the cube facing up
<i>R</i>	side of the cube facing right
<i>F</i>	side of the cube facing front
<i>L</i>	side of the cube facing left
<i>B</i>	side of the cube facing back
<i>D</i>	side of the cube facing down
<i>U1...U9</i>	stickers of the side <i>U</i> proceeding in reading direction

$R1...R9$	stickers of the side R proceeding in reading direction
$F1...F9$	stickers of the side F proceeding in reading direction
$L1...L9$	stickers of the side L proceeding in reading direction
$B1...B9$	stickers of the side B proceeding in reading direction
$D1...D9$	stickers of the side D proceeding in reading direction

1. INTRODUCTION

The Rubik's Cube is a worldwide known puzzle game released in 1974 by Hungarian architecture professor Ernő Rubik [1]. The idea of the puzzle is to order all the pieces of the cube in a way that each of the six faces consists only of pieces that match in color with each other. Since its release, the Rubik's Cube has been seen as a tool to measure and teach skills of problem solving and three-dimensional perception [1].

As machine learning and computer vision tools have developed in recent decades, robotic solutions for the cube have been developed [2–4]. Solving the Rubik's Cube is an interesting task from the robotics' point of view since it demands accurate color detection to detect the state of the cube, effective mathematical computing algorithm to find a solution, and precise hardware to implement the moves from the solution. In this thesis, however, the hardware related issues are not considered since human user will be operating the Rubik's Cube.

The purpose of this thesis is to create a computer program that detects the state of a scrambled Rubik's Cube via webcam, computes a solution for it, and guides the human user to execute the solution. The focus will be on the computer vision task of detecting the state of the cube rather than the mathematical problem of solving the cube. The algorithm solving algorithm is taken as a ready-made function from open-source programming library called Kociemba [5]. Besides creating the Rubik's Cube solver program, the goal of this thesis is to examine commonly used computer vision tools and discuss the bearing of computer vision on modern robotics.

The report is divided into five chapters, the first one being this introduction. In chapter 2, the relevant background information for the topics of this thesis is represented. The Rubik's Cube itself, evolution and uses of computer vision, and already existing robotic solutions for the cube will be discussed. In chapter 3, the implementation of the program is presented. Functioning of the program and the tools and algorithms used will be discussed. In chapter 4, the results of the programming are presented and evaluated. It will be discussed, whether the purpose of this thesis was achieved, what were the biggest challenges during the work, and what kind of improvements or future applications there could be for this program. Chapter 5 will be the conclusion of the thesis.

2. BACKGROUND FOR CUBE DETECTION AND SOLUTION COMPUTING

The problem of solving the Rubik's Cube with computer can be roughly divided into two parts. The first part includes all the problems related to detecting the cube and its current state, meaning the positions and orientations of all the pieces, via webcam. The second part has to do with the mathematical problem of finding a solution, meaning a series of turns to be made to solve the cube. To develop a program that can solve both parts of the problem it is important to understand what kind of an object the Rubik's Cube is, how computer vision, and especially object detection, works in general, and how solutions for a given state of the cube can be computed. It is also helpful to know about the robotic solutions for the Rubik's Cube that have already been made. The purpose of this chapter is to offer background information about these topics to better understand the choices made during the coding process, that are described in chapter 3.

2.1 The Rubik's Cube

As mentioned in introduction, the Rubik's Cube is a puzzle invented in 1974 by architecture professor Ernő Rubik, who invented it to teach his students spatial visualization and problem solving [1]. The problem of solving the cube is indeed not trivial since according to estimations less than 5.8 percent of people can solve it [6]. In this subchapter the basic structure of the cube and its mechanics are discussed to get an idea of what kind of an object it is and how can it be modelled. After that, the mathematics of the solution computing and the history of solving algorithms are briefly discussed.

2.1.1 The structure and mechanics of the cube

The cube consists of six center pieces, twelve edge pieces, and eight corner pieces as represented in figure 1. The center pieces cannot be moved in relation to each other and thus, they determine the final color of each face. Positions of edge and corner pieces can be changed by turning the sides of the cube and the idea of the puzzle is to find the correct turns to be made to get stickers of each face to match with each other in color.

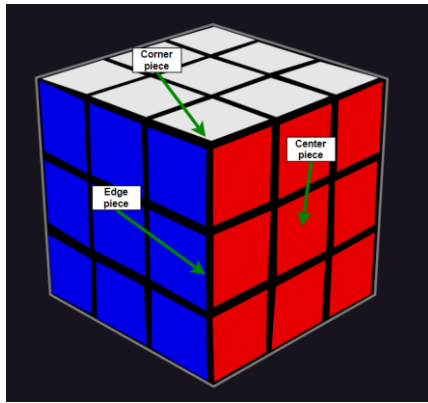


Figure 1. The terminology of the cube pieces.

			Wh1	Wh2	Wh3						
			Wh4	Wh5	Wh6						
			Wh7	Wh8	Wh9						
Re1	Re2	Re3	Bl1	Bl2	Bl3	Or1	Or2	Or3	Gr1	Gr2	Gr3
Re4	Re5	Re6	Bl4	Bl5	Bl6	Or4	Or5	Or6	Gr4	Gr5	Gr6
Re7	Re8	Re9	Bl7	Bl8	Bl9	Or7	Or8	Or9	Gr7	Gr8	Gr9
			Ye1	Ye2	Ye3						
			Ye4	Ye5	Ye6						
			Ye7	Ye8	Ye9						

Figure 2. The representation of the state of the cube.

As can be seen from figure 1, each corner piece consists of three stickers of different colors and each edge piece consists of two stickers of different colors. Each face of the cube consists of nine stickers. The state of the cube can be represented as locations and orientations of all the pieces or simply as the colors of the stickers surrounding each center piece. The advantage of the latter way of representation is that it makes it possible to detect the state of the cube sticker by sticker one side at a time without taking into consideration of which piece a given sticker belongs to. This way of representing the state of the cube is shown in figure 2, where symbols *Wh*, *Re*, *Bl*, *Or*, *Gr*, and *Ye* stand

for six colors of the cube: white, red, blue, orange, green, and yellow. Since the center pieces cannot be moved in relation to each other, stickers in positions *Wh5*, *Re5*, *Bl5*, *Or5*, *Gr5*, and *Ye5* will always be the same and thus, it is practical to name all the nine sticker positions of each face after the center color of the face.

2.1.2 Computing solutions

As stated before, solving the Rubik's Cube is not a trivial task. It took mathematicians more than twenty years to develop an optimal solving algorithm after the cube was first released in 1974. The optimal algorithm is named Korf's algorithm, after US computer scientist Richard Korf, who developed it in 1997. The word optimal refers to the algorithm's ability to find shortest possible solutions to arbitrary states of the cube. After Korf's optimal algorithm was introduced in 1997, it took another 13 years to prove that the minimum number of moves required to solve the cube from any starting position is 20, when both a quarter turn and a half turn of any face is considered as one move. The optimal algorithm and the minimum number of turns to solve any starting position of the cube are often referred to as God's algorithm and God's number. [7]

Korf's algorithm was built upon the ideas of German mathematician Herbert Kociemba, who introduced his near-optimal algorithm in 1992. Kociemba's algorithm can find solutions of around 20 moves for random positions of the cube but there is no guarantee that these solutions are optimal. However, Kociemba's algorithm is significantly faster and requires much less computing power than Korf's algorithm. [7] For this reason, it can be justified to use Kociemba's algorithm in a program that is designed to run on average consumer hardware with relatively low computing power and where it is not crucial to use only optimal solutions.

2.2 Computer vision as a crucial part of robotics

Computer vision refers to a field of artificial intelligence (AI) which includes the computer's capabilities of processing information from still or live image [8]. The most relevant component of computer vision considering this thesis is object detection. It is a sub-field inside computer vision and includes all the tasks of detecting instances of visual objects, like humans or cars, from image inputs [9]. Vision is usually referred to as the most important sense of humans [10][11]. It is seen as the dominant way of getting information of one's environment. Thus, it seems logical to value computer vision as a crucial feature, when aiming to develop robots that can operate independently in their working environments and perform human-like tasks.

2.2.1 Uses

There are many uses for computer vision in number of fields of technology. Computer vision is used in autonomous vehicles, facial recognition, augmented reality environments, and healthcare applications to name a few. It is used both for entertainment purposes and making different working tasks safer, more efficient, and in some cases completely automated.

Computer vision is a key technology behind self-driving vehicles and autonomous mobile work machinery. Companies like Tesla have been developing autonomous driving features for their cars for the past decade. A full autonomy on a car driving among the traffic is still a work in progress, but there are already many automation systems assisting the human driver that work based on the visual environment of the car. [12]

While human drivers are still needed for the cars on the public roads, there are already many applications using higher degrees of automation in working sites. Kalmar's automated straddle carriers [13] and Sandvik's AutoMine system [14] are examples of usage of mobile work machinery operating without driver. Autonomous driving and operating can be implemented safely with these machines working on ports and mines since there is no unpredictable traffic.

There are also many features utilizing computer vision in smart phones that people use in everyday life. In November 2017 Apple introduced Face ID as a new feature of iPhone X. It enables users to unlock their phone, log in to different accounts, or make payments with the phone without using passwords just by showing their face to the camera. Fast and accurate facial recognition is possible due to advanced image processing techniques done with depth camera and neural processing units. [15] In recent years many other facial recognition applications by different companies have been released and the new technology is widely adopted by the users: most people aged 18–34 in US is using facial recognition applications daily [16].

Computer vision has also important uses in the field of healthcare. It can be used as a tool for medical image analysis, for example, to detect tumors. A study from 2020 compared an AI based detection system to human experts in detecting breast cancer from medical images. The AI system ended up outperforming the human experts by giving more accurate diagnosis [17]. There is also ongoing research and development of using computer vision tools in surgeries to help surgeons to make better decisions and prevent errors [18][19][20].

Based on the examples represented above, it can be stated that computer vision is an important discipline considering the modern society and its development. It can be widely

used in different fields of activity, and it will probably be one of the key technologies behind automating tasks that, for now, require human operators.

2.2.2 Evolution of object detection

Object detection is seen as highly important task in the field of robotics today since many of the visual tasks have to do with detecting instances from the visual environment. Thus, object detection techniques have been studied and developed a lot in recent years. It is shown in figure 3, how the number of scientific publications in object detection has rose in past twenty years.

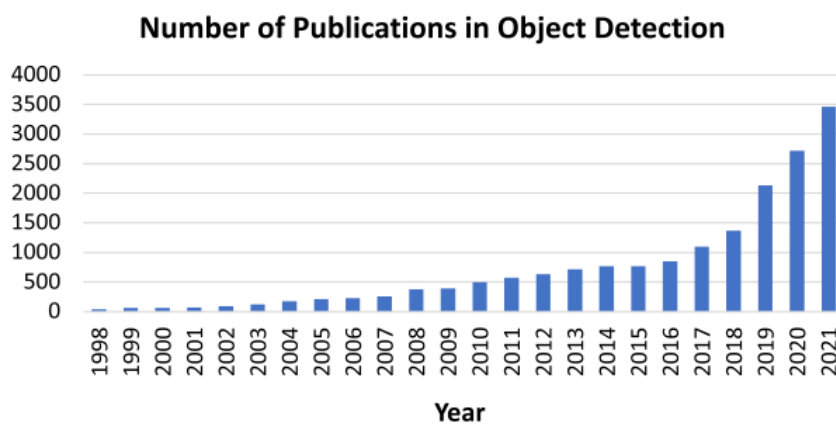


Figure 3. Number of publications in object detection from 1998 to 2021 [9].

After plateauing for years, the evolution of object detection went through an aggressive acceleration phase in early 2010's due to rise of deep learning. Deep learning refers to a kind of machine learning, where information processing happens using deep convolutional neural networks (CNNs), which mimic the structure and functioning of human brain. Deep learning techniques improved rapidly in 2010's enabling revolutionary improvements in all the fields of the AI, including object detection. Since techniques to train computers with huge amounts of data were discovered, the success of detecting objects in constantly changing circumstances has improved significantly. [9][21, chapter 1].

The deep learning revolution has made it possible to create artificial systems that already outperform humans in some visual tasks, like recognizing different objects from images [22]. However, it is good to remember that the conclusion of computer vision being superior to human vision in general should not be drawn from individual cases where AI beats humans in a specific task. Mastering contextual understanding and handling of ambiguity is still a challenge with computer vision. It is also good to keep in mind that the

performance of the most accurate artificial visual systems seems not to correlate well with performance of human vision [22]. This means that artificial systems find different kinds of circumstances challenging and make errors for different reasons than humans do. Because of this, the visual errors made by AI systems can be unpredictable and therefore cause unexpected actions in autonomous machines. For this reason, it is probably a good idea to have some degree of human supervision behind autonomous machines operating in places where they can cause serious damage.

2.2.3 Tools and platforms

There are many programming languages, tools, and platforms where to choose when starting a computer vision project. Although there are many options available, the developer community seems to value a few choices over others. Pretty common view seems to be that Python is one of the best languages of choice in the context of AI and machine learning projects. It is ranked high because it is simple and easy to learn, has lots of versatility, and includes an extensive collection of libraries to be used with AI. [23][24]

Another programming languages worth mentioning are C++ and MATLAB. When comparing C++ and Python in the context of machine learning, and computer vision, C++ has advantages in the speed of code execution but with Python converting ideas to code and prototyping is seen easier and faster. Also, the great availability of state-of-the-art tools and frameworks is seen as advantage for Python. [23][24]

MATLAB, on the other hand, has been widely used in academic institutions. It has extensive and clear documentation and debugging of the code is seen easy compared to other languages. MATLAB has relatively good capabilities in image processing and machine learning, but it is clearly outperformed by the best computer vision libraries that can be used with Python or C++ in the efficiency of code execution. Also, compared to open-source libraries of Python and C++, MATLAB is not so popular choice outside the academic institutions since users need to buy a license to use it. [23][24] Considering that a significant part of the developer community operates outside the academic institutions, the support for the community in the form of tutorials, examples, and discussion forums, might be seen as crucial advantage for the open-source platforms. For the reasons mentioned above, Python was the language of choice for the programming work of this thesis.

As mentioned above, there are number of software libraries and packages that can be used in computer vision and machine learning projects. Some of the most popular ones are OpenCV, TensorFlow, PyTorch, and Scikit-learn [25–32]. While TensorFlow and PyTorch specialise in powerful deep learning, and Scikit-learn is known as beginner-

friendly choice for machine learning, but with more limited performance [27][28], OpenCV is often mentioned as the first choice for computer vision tasks specifically [29–32].

OpenCV is an open-source software library that includes more than 2500 algorithms for computer vision and machine learning tasks. Both classic and state-of-the-art tools can be found inside the library. OpenCV supports multiple programming languages such as Python, C++, and MATLAB. It also has large user community, and it is widely used in companies and research groups. [33] Due to its excellent computer vision tools and great community support, in the form of online tutorials and examples, it was a natural choice for the programming work of this thesis. It works well with Python and has all the needed tools to offer for the Rubik’s Cube detection.

2.3 Existing robotic solutions for the Rubik’s Cube

There already exists many robotic solvers for the Rubik’s Cube. In this subchapter, three of these will be discussed because they are somehow remarkable.

2.3.1 Guinness World Record robot from MIT

The first robot to represent holds the Guinness World Record in the fastest time of robot solving the Rubik’s Cube [2][34]. The video, where the cube is solved in just 0.38 seconds was released in March 2018 on YouTube [35]. The robot was built by two MIT students Ben Katz and Jared Di Carlo. It consists of six servo motors, connected to each face of the cube, their custom drivers, and two PlayStation Eye cameras, directed towards the opposite corners of the cube in order to detect the starting position all at once [36][37]. Just like in the work of this thesis, the processing happens on Linux based computer, OpenCV library is used for color detection, and Kociemba’s algorithm, although differently fine-tuned than the one used in this thesis, is used to compute the solution [36][37]. This robot is remarkable in the field of Rubik’s Cube solvers since it is still the fastest one to solve the cube.

2.3.2 One-handed cube solver from OpenAI

Another interesting Rubik’s Cube solver is OpenAI’s system called Dactyl that controls a robotic hand that solves the cube. Solving the cube one-handed is hard task even for humans and requires a lot of dexterity. This solver is not meant to be the fastest in solving the cube but rather a step towards general-purpose robots that can perform human-like tasks widely.

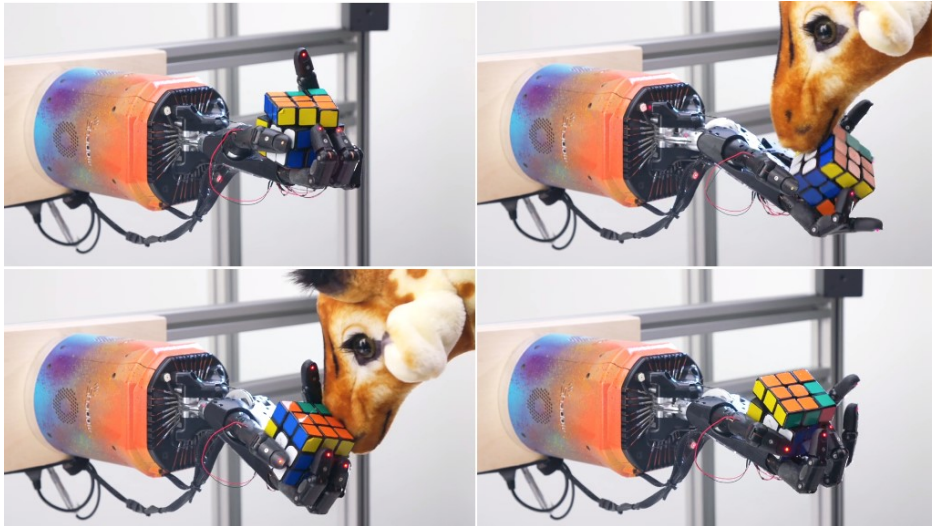


Figure 4. Distracting OpenAI's Rubik's Cube solver with stuffed animal [3].

This solver uses Kociemba's algorithm to find the solution like the MIT world record solver or the solver created in this thesis, but the executions of specific moves are not hard-coded. Handling of the cube is possible due to reinforcement learning with neural networks. During the training in simulation environment a method called automatic domain randomization (ADR) is used to prepare the robot to operate in diverse situations of the real world. This means automatically increasing the randomness of the environment every time the neural network reaches a performance threshold. Due to this method of training the robot is capable of handling kind of situations in a real world that it has never seen before. For example, the robot is able to solve the cube correctly even though it is distracted with a stuffed giraffe, as shown in figure 4. [3] This ability to react to stimulus never seen during the training and the dexterity of the hand make this robot remarkable. It is clearly more human-like and general-purposed than single task oriented industrial robots we are used to seeing.

2.3.3 Gan Robot

The third Rubik's Cube solver to discuss is Gan Robot [4], a product developed and launched by speed cube brand Gan. The robot is represented in figure 5. It differs from the two robots represented earlier by being a commercial product rather than a research project.

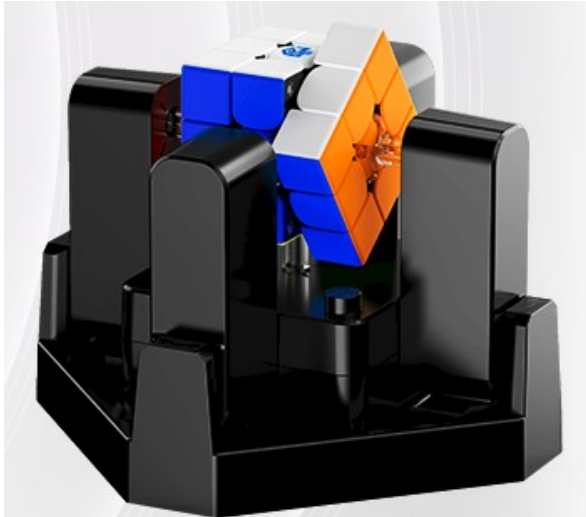


Figure 5. Gan Robot [4].

The robot needs to be used with a smart cube from the same brand. Both the cube and the robot need to be connected to a smart phone via Bluetooth allowing user to see the position and the moves of the cube on the screen of the phone. This robot does not use cameras to detect the position of the cube since the smart cubes are built in a way that they know their own initial state. Since the robot solver and the smart cube are both connected to a smart phone, they can share information with each other enabling the robot to solve the cube using the information of the initial state of the cube and the solution computed by the smart phone. This robot is worth mentioning since it is the first Rubik's Cube solving robot that is mass produced and designed for consumers. [4]

3. IMPLEMENTATION

In this chapter, the actual program created as a part of this thesis, and the process of developing it, will be discussed. The functioning of the program, including the tools and methods used and developed as part of the process, will be explained. To summarize from chapter 2, the program is coded in Python, OpenCV software library is used for image processing and object detection, and the solution computing for given cube positions happens using Kociemba's algorithm, found in the software library called Kociemba.

3.1 Interaction with the user

The interaction between the user and happens through video input from the webcam, video output from the computer screen, and a few keyboard commands that user can give during the running of the program. After the program is started from the terminal or integrated development environment (IDE), a window displaying the modified video input from the webcam appears and from this point on, this window is everything that user needs to see. At this point, the user should see live image of him-/herself holding the Rubik's Cube at the front of the computer.

Different instruction texts will appear on the top part of the window during the running of the program always telling the user what to do next. The user is constantly told, which side of the cube should be shown to the camera using the text instructions on top of the window. The user is also guided with arrows drawn on the screen to rotate the cube in desired directions. During the detection phase, the user is supposed only to rotate the cube showing all the six faces to the camera in predetermined order, but not to make any turns. The view during the detection, when the user is asked to show the red face of the cube is represented in figure 6. Since the blue face is facing to the camera (and the white face is facing up) the cube should be rotated left to show the red face to the camera. That is what the arrows are supposed to tell the user enabling him/her to operate fast and effortlessly without glancing the text instructions all the time. The text instructions are still constantly available in order to unambiguously express which face should be shown.

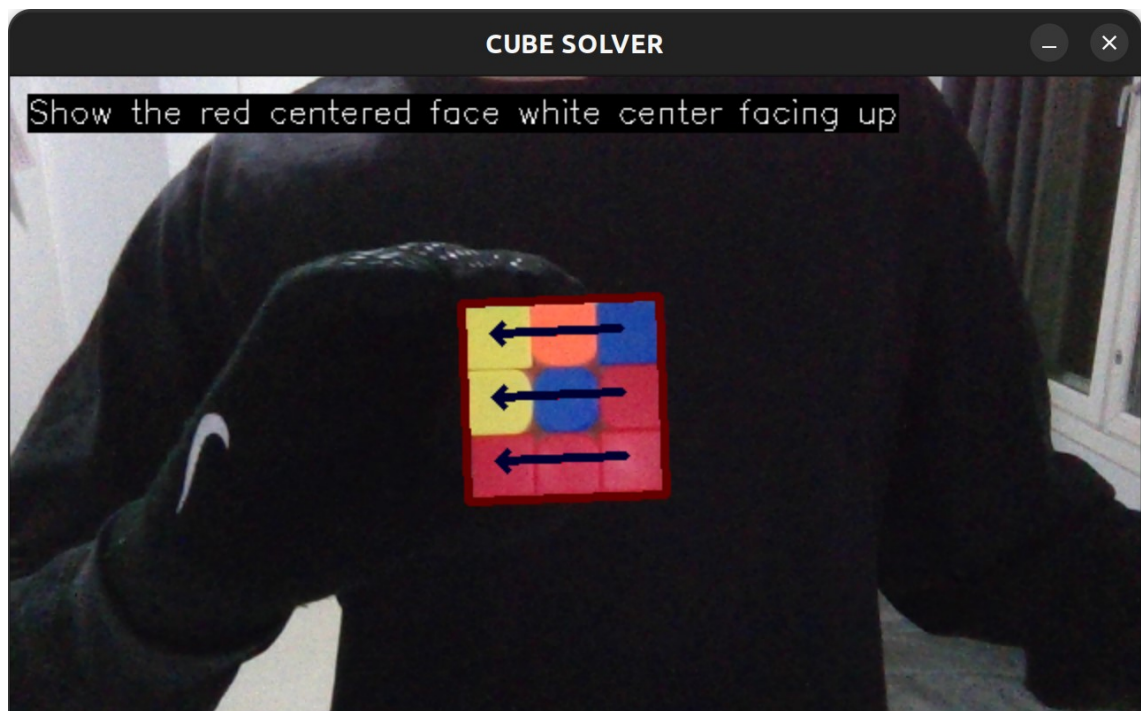


Figure 6. Guiding the user to rotate the cube counterclockwise.

At the end of the detection of each face, the program draws a model of the detected face on the top right corner of the window enabling user to verify that the detection was successful before saving the detected face on the model of the cube inside the program. Verification is done by pressing either 'y' for yes or 'n' for no, when asked if the face was correctly detected. This situation is represented in figure 7.



Figure 7. Asking the user if the cube was detected correctly.

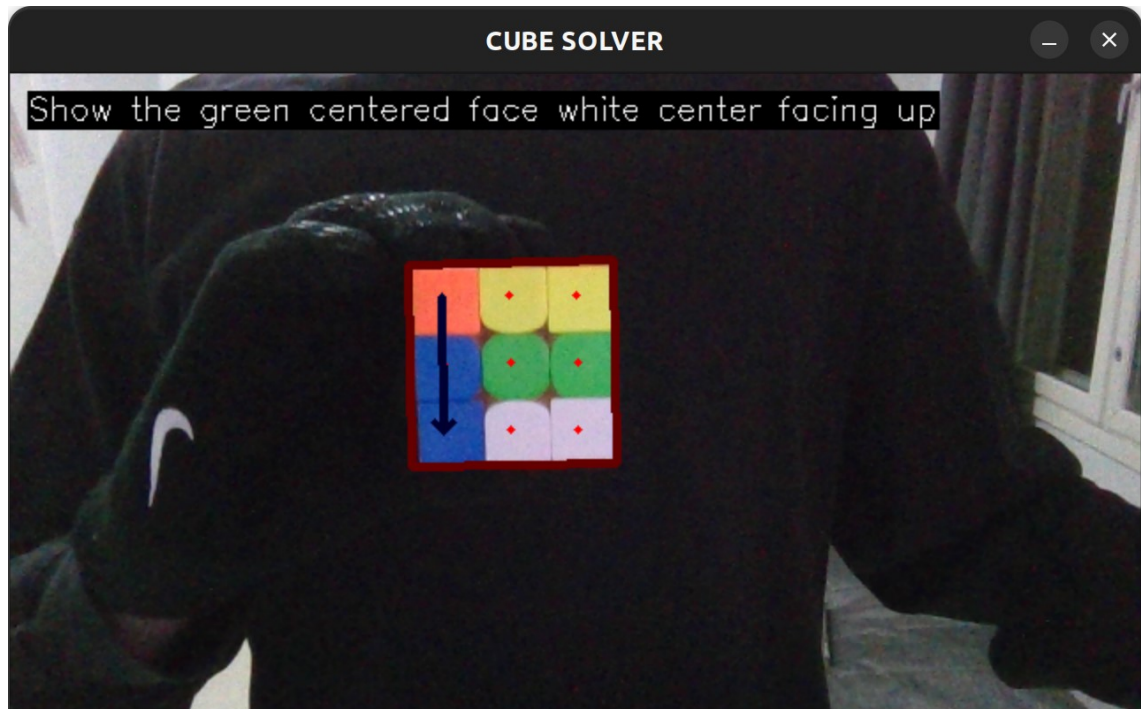


Figure 8. Guiding the user to turn the left side of the cube.

During the solving phase, similar arrows that were used during the detection are used to guide the user to turn individual sides of the cube. A situation where the user is guided to turn the left side of the cube during the solve is represented in figure 8.

3.2 The model of the cube

In the program, the Rubik's Cube is modelled as a Python class called `Cube`. The state of the cube is represented as a dictionary where the center colors of the faces are the keys, and the corresponding values are 3x3 arrays where each item represents a color of the sticker on that location on the face. In program 1, a state of a solved cube is represented using this data structure. Figure 2 represented in subchapter 2.1.1 is basically a visualization of this structure. This way of representing the state of the cube is unambiguous since the center pieces cannot be moved in relation to each other, as mentioned earlier.

```

self.state = {

    "white": [
        ["white", "white", "white"],
        ["white", "white", "white"],
        ["white", "white", "white"]
    ],

    "orange": [
        ["orange", "orange", "orange"],
        ["orange", "orange", "orange"],
        ["orange", "orange", "orange"]
    ],

    "blue": [
        ["blue", "blue", "blue"],
        ["blue", "blue", "blue"],
        ["blue", "blue", "blue"]
    ],

    "yellow": [
        ["yellow", "yellow", "yellow"],
        ["yellow", "yellow", "yellow"],
        ["yellow", "yellow", "yellow"]
    ],

    "red": [
        ["red", "red", "red"],
        ["red", "red", "red"],
        ["red", "red", "red"]
    ],

    "green": [
        ["green", "green", "green"],
        ["green", "green", "green"],
        ["green", "green", "green"]
    ]
}

```

Program 1. *The data structure of the state of the cube.*

Cube class includes a method for computing a solution for a given state of the cube. Inside the method, Kociemba's algorithm, which can be found as ready-made function inside Kociemba library [5], is used for computing solutions. The interaction with Kociemba's solve-function is described more precisely in subchapter 3.4.

Cube class also includes separate methods for turning each of the six faces of the cube. Each face can be turned either clockwise or counterclockwise. When a turn method is called for some of the faces of the *Cube* object, each face will be updated corresponding to the turn. By calling the turn methods of different faces the *Cube* object in the program can be manipulated just as an actual cube in the real world.

3.3 Detection

Detecting the cube correctly turned out to be the most challenging task to perform during the coding process. The process of detection can be divided into three phases. First, the cube needs to be distinguished from the background as a whole. Second, all the nine stickers of the face showing are located. This happens by calculating the locations of their center points based on the outline of the cube. Third, the contours of all the white, blue, orange, yellow, red, and green shapes are found from the image and the colors of the stickers are concluded based on inside which contour the center point of each sticker is located. Finally, the verification is asked from the user to know if the detection was completed correctly. If the verification is denied, the whole process starts again, and this will go on until the user verifies that the face is correctly detected.

To be more specific, the phases one, two, and three are iterated rapidly inside a loop multiple times per second processing a single frame from the live image on each iteration. Each time a color of a sticker is detected, it will be saved into an array that represents the face being detected. When values for all of the nine stickers are saved, the face is considered to be detected and the verification is asked from the user. After the verification, the face is saved into the Cube object and the program moves on to detect the next face.

3.3.1 Distinguishing the cube from the background

As mentioned above, the first thing that needs to be done successfully in order to detect a given face of the cube is to detect the cube as a whole from the background. This needs to be done so that the stickers on the face can be located and after that their colors can be detected. During the coding process, distinguishing the cube from the background turned out to be quite challenging without setting some constraints for the environment and for the user. This is because in many cases, the colors of the user's skin and clothes and the room and the furniture at the background have overlap with the colors of the cube. In other words, the six colors of the cube tend to appear also in the background, which makes the cube harder to detect from the image.

The problem of distinguishing the cube from the background is solved by making some requirements for the environment and the way that the cube needs to be shown to the camera. First of all, the immediate background behind the cube should be black or as dark as possible. This means that other colors are allowed to appear on the sides of the image, but the cube should be surrounded by dark background, which does not overlap with the colors of the cube. In practice this can be done as simply as wearing a black shirt and directing the webcam in a way that the shirt will be the immediate background

behind the cube as shown in all the figures representing screen shots from the running program. Also, wearing black gloves is necessary if the color of the user's skin overlaps with the colors of the cube. When this requirement is fulfilled, the cube can be distinguished from the background by using a mask that blocks black and other really dark shades from the image but lets other colors through as white. This kind of mask is represented in figure 9 where the original input image is displayed on the left and the masked image is displayed on the right.

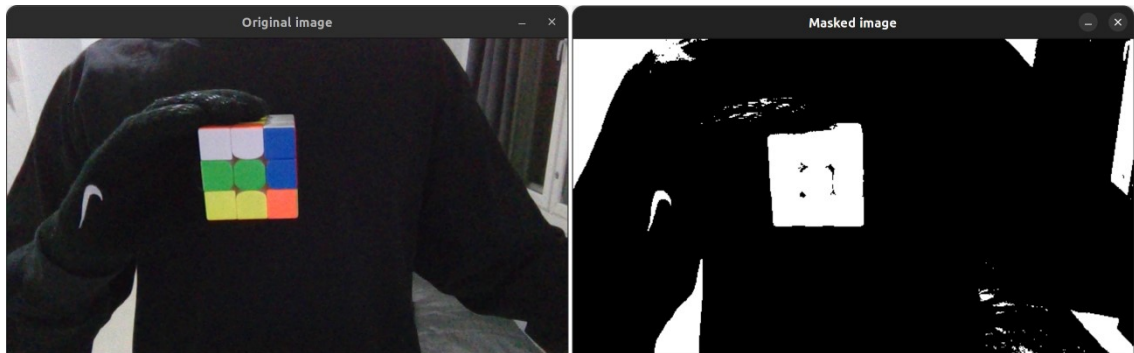


Figure 9. Filtering out dark colors.

The second requirement is that the cube needs to be held at an appropriate distance from the camera so that really small and really big objects appearing in the image can be ignored when finding the cube. This is done by using an OpenCV function that is designed to find all the contours from a given mask [38]. This means finding the borderlines of all the shapes that get through the mask and are not touching with each other. After finding all the contours their areas are calculated and the ones with area less than 10 000 pixels or more than 40 000 pixels are ignored. Making this requirement seems reasonable since the user would probably be holding the cube at a valid distance (about 20–35 cm from the camera) anyway. The situations where the cube is held at the minimal and maximal distances are represented in figure 10. As can be seen from the figure, if the cube is detected from the image, a red outline will appear around it and red dots will be drawn on the sticker centers. Note that the colors of the stickers have not yet been detected in any of the screen shots and therefore the face drawn on the top right corner is gray.

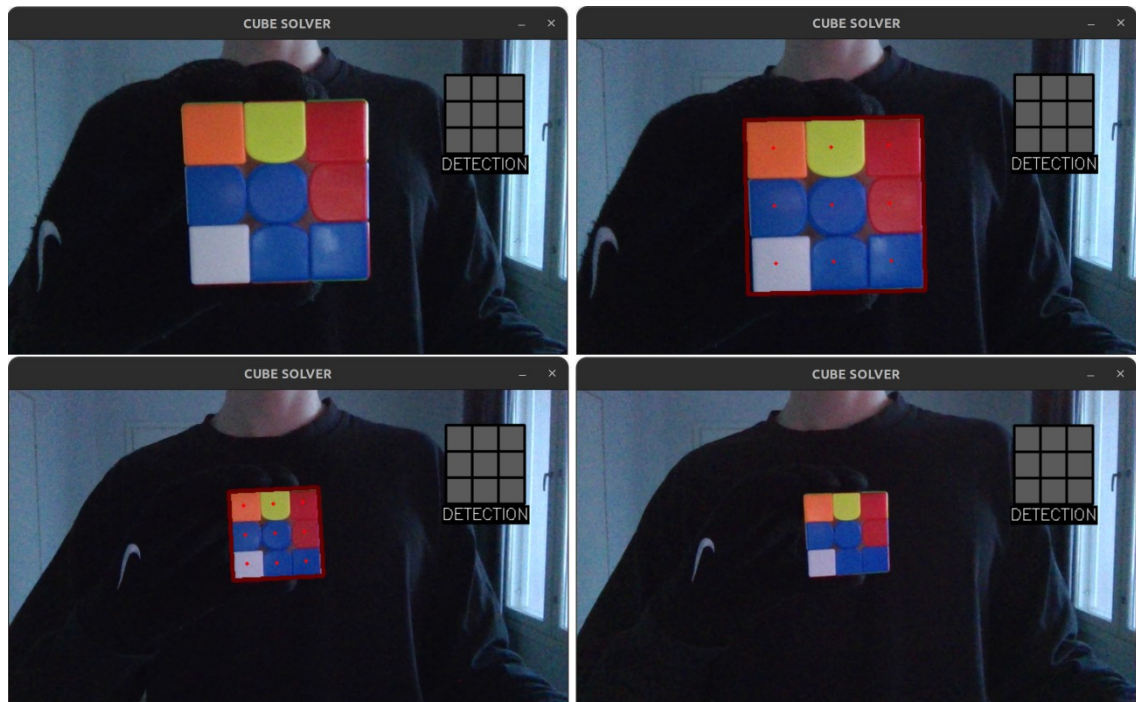


Figure 10. Minimum and maximum distances from the camera. Top left: too close, top right: at the minimum distance, bottom left: maximum distance, bottom right: too far.

The third requirement is that the cube needs to be held relatively close to the center of the image. This allows the program to ignore the irrelevant objects on the sides of the image. In practice, this means that all the contours that have their center of mass further than fifty pixels away from the image center are ignored. This gives the user enough clearance to handle the cube naturally but at the same time makes it possible to ignore all the objects from the background that have the same colors as the cube. This requirement can also be seen as reasonable since it is natural to hold the cube approximately at the center of the image anyway.

The fourth requirement is that the cube cannot be arbitrary rotated when it is shown to the camera. The cube should be shown to the camera one face at a time holding it in close to an upright position. Some rotation of the face showing is allowed, but the deviation from the upright position should not be more than 30 degrees so that it stays clear which side of the cube is facing up. It is crucial to know which side is facing up when the face shown to the camera is being detected in order to make unambiguous interpretations of the state of the cube. The maximal rotations allowed during the detection are represented in figure 11. As in figure 10, the red outline the red dots on the sticker centers appear when the cube is detected from the image.



Figure 11. Maximal rotations allowed. Top left: too much counterclockwise rotation, top right: maximal counterclockwise rotation, bottom left: maximum clockwise rotation, bottom right: too much clockwise rotation.

The final thing the program checks from the potential contour of the cube is its shape. The shape of the contour outlining the cube in 2D image should be approximately square. Checking the shape of the contour happens by comparing the original contour to the minimum area rectangle surrounding it, which can be found using an OpenCV function designed for this purpose [39]. The original contour and the corresponding minimum area rectangle are represented in figures 12 and 13. If the object outlined by the original contour is, in fact, the cube, the original contour and the smallest possible rectangle surrounding it should be almost identical. This is checked by calculating the distance of each point of the original contour to the surrounding rectangle. If any of these distances is larger than the margin of error, in this case twenty pixels, the object outlined is not considered to be the cube. If this test is passed, the height and width of the rectangle are compared to each other to make sure that the rectangle is approximately square. If the difference between the width and the height is larger than the margin of error, also twenty pixels, the object is once again not considered to be the cube.

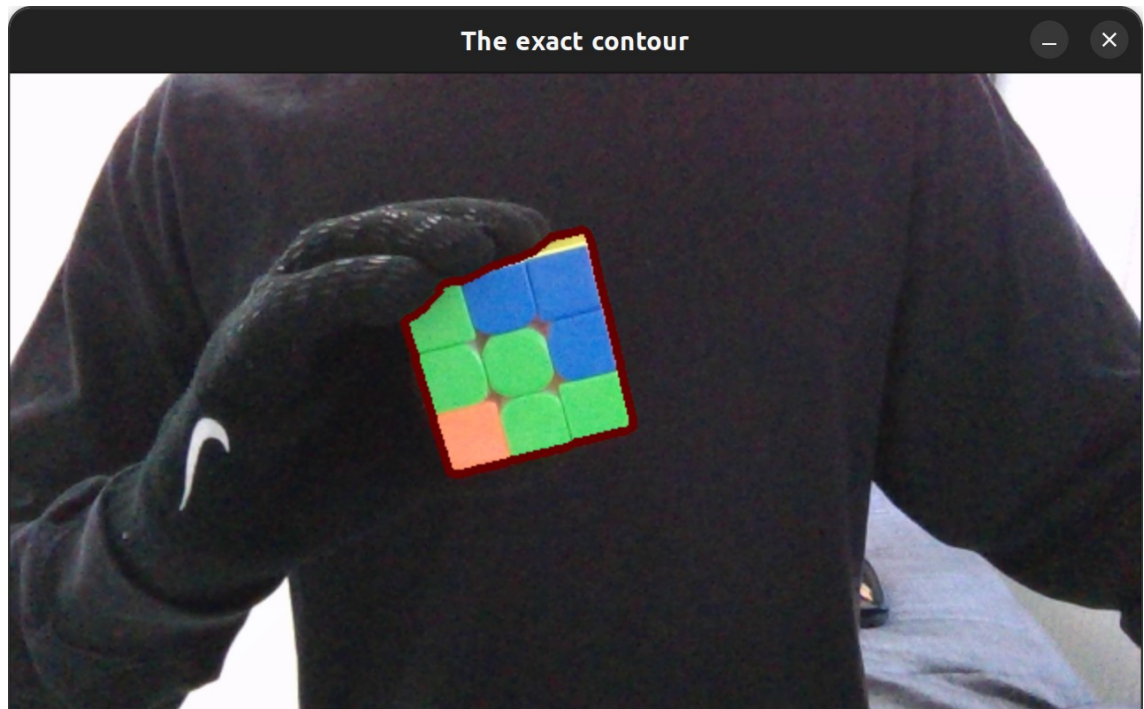


Figure 12. *The exact contour of the detected cube.*

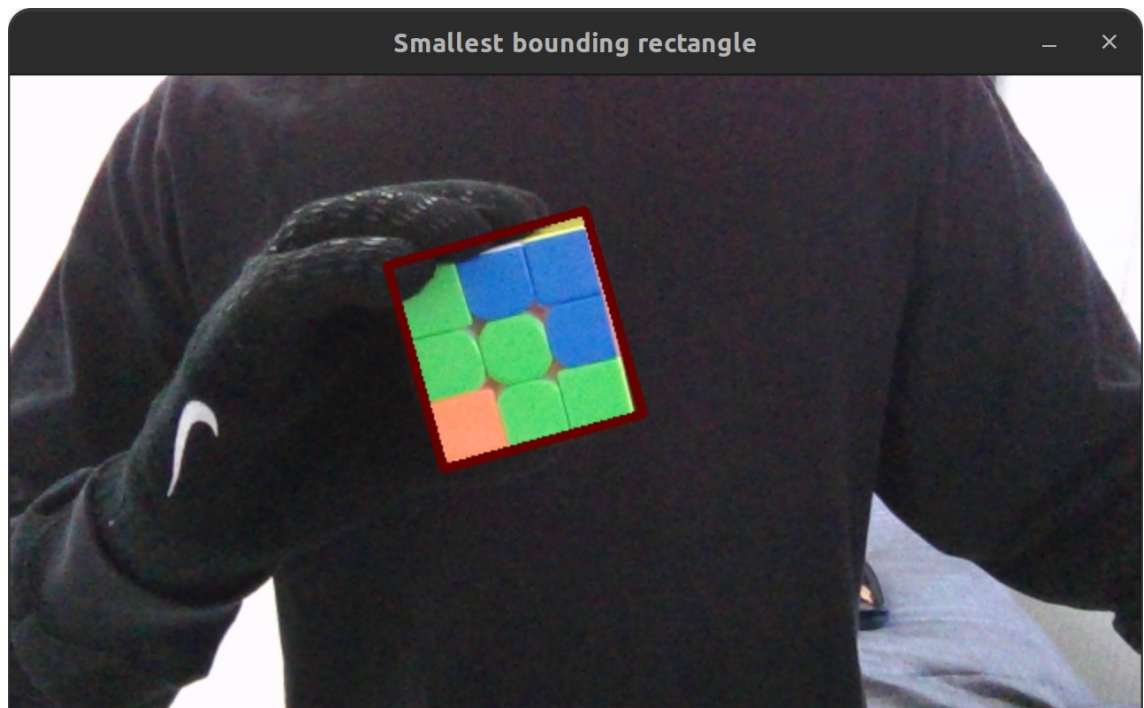


Figure 13. *The smallest rectangle that can be drawn around the detected cube.*

To conclude, the Rubik's Cube is detected from an input image by first filtering out all the dark colors from the image enabling objects colored like the cube to distinguish from the dark background, then finding all the contours of separate shapes from the filtered image, and then checking if any of these contours is correct sized, located at the center of the image, square shaped, and not rotated more than 30 degrees from the upright position. A contour that meets all the requirements is considered to be the outline of the cube. Although this sounds like a lot of work to do, the cube seems to be detected immediately after its appearance on the screen from the user's point of view, as long as the limit values set for the colors are set properly.

3.3.2 Locating the center points

After the outline of the cube has been detected from the image, the next thing to do is to calculate the locations of the center points of all the nine stickers of the face shown to the camera. This can be done when width w , height h , and rotational angle θ of the rectangle outlining the cube are known. Measures w , h , and θ are represented in figures 14 and 15.

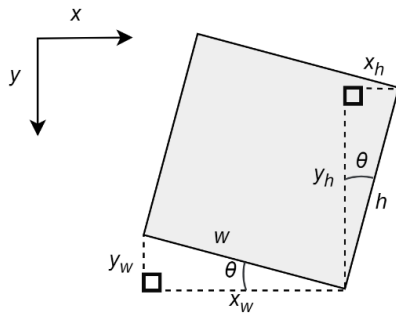


Figure 14. Width w , height h , and rotational angle θ when $0^\circ < \theta \leq 30^\circ$.

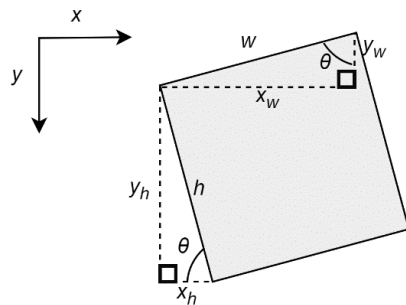


Figure 15. Width w , height h , and rotational angle θ when $60^\circ \leq \theta \leq 90^\circ$.

As can be seen from figures 14 and 15, rotational angle θ is the angle between the horizontal plane and one side of the rectangle. The side of the rectangle that is being used in the calculation is the one between the corner with the highest y -value and the next corner from it in clockwise direction. Note that using OpenCV coordinates x -values grow when going right and y -values grow when going down. As mentioned before, it needs to be clear which side of the cube is facing up and this is why the cube is not allowed to be rotated more than 30 degrees from the upright position. Using OpenCV notation this means denying the rotational angle values between 30 and 60 degrees.

Variables x_w and y_w stand for x - and y -components of width w of the rotated cube, and correspondingly, variables x_h and y_h stand for x - and y -components of height h of the rotated cube. Using basic trigonometric functions, transitions in directions x and y can be represented as functions of transitions parallel to width and height of a rotated cube. Let's consider both cases, the one where the rotational angle is between 0 and 30 degrees and the one where it is between 60 and 90 degrees. With the use of sine and cosine functions, x - and y -components of width w can be represented as

$$y_w = \begin{cases} \sin(\theta) w & | 0^\circ < \theta \leq 30^\circ \\ \cos(\theta) w & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (1)$$

and

$$x_w = \begin{cases} \cos(\theta) w & | 0^\circ < \theta \leq 30^\circ \\ \sin(\theta) w & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (2)$$

Correspondingly, x - and y -components of height h can be represented as

$$y_h = \begin{cases} \cos(\theta) h & | 0^\circ < \theta \leq 30^\circ \\ \sin(\theta) h & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (3)$$

and

$$x_h = \begin{cases} \sin(\theta) h & | 0^\circ < \theta \leq 30^\circ \\ \cos(\theta) h & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (4)$$

When locating the sticker centers, the top left corner of the outline of the cube is used as the origin point in the loop where the localization happens. Thus, its coordinates are marked as x_0 and y_0 , as can be seen from figure 16. The localization of the center points happens by calculating x - and y -transitions from the origin point to each of the sticker

centers. The first one to be located is the sticker at the top left position. From there, the location happens inside the loop row by row going from left to right.

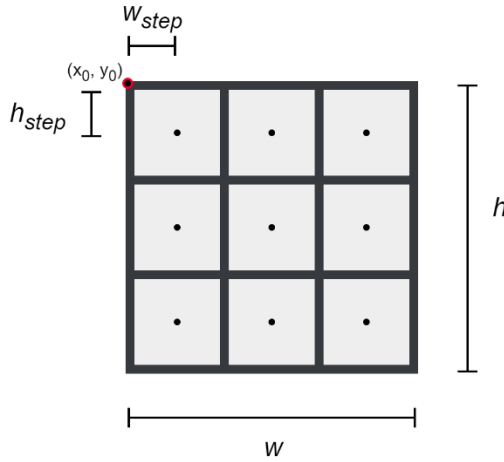


Figure 16. Step sizes during iteration.

As represented in figure 16, variable w_{step} is the distance from the side of the sticker to the center of the sticker. Correspondingly, variable h_{step} is the distance from the top of the sticker to the center of the sticker. The reason for the subscript *step* in the variable names is that these variables represent the step sizes that are used in the loop when mapping the cube. As can be seen from figure 16, these are the largest step sizes that can be used when moving from the origin point at the top left corner of the cube outline to each sticker center. The values of w_{step} and h_{step} can be calculated as

$$w_{step} = \frac{w}{6} \quad (5)$$

and

$$h_{step} = \frac{h}{6} . \quad (6)$$

Now, using equations 1–4, but substituting w_{step} for w and h_{step} for h , and taking into consideration that the steps in width-direction are taken from left to right and the steps in height-direction are taken from top to bottom, the transitions in directions x and y corresponding to the steps can be represented as

$$dy_w = \begin{cases} \sin(\theta) w_{step} & | 0^\circ < \theta \leq 30^\circ \\ -\cos(\theta) w_{step} & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (7)$$

$$dx_w = \begin{cases} \cos(\theta) w_{step} & | 0^\circ < \theta \leq 30^\circ \\ \sin(\theta) w_{step} & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (8)$$

$$dy_h = \begin{cases} \cos(\theta) h_{step} & | 0^\circ < \theta \leq 30^\circ \\ \sin(\theta) h_{step} & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (9)$$

and

$$dx_h = \begin{cases} -\sin(\theta) h_{step} & | 0^\circ < \theta \leq 30^\circ \\ \cos(\theta) h_{step} & | 60^\circ \leq \theta \leq 90^\circ \end{cases} \quad (10)$$

The equations for total transitions dx and dy , that take into account both the steps in width-direction and the steps in height-direction, are

$$dy = ndy_w + mdy_h \quad (11)$$

and

$$dx = ndx_w + mdx_h \quad (12)$$

where n is the number of steps in width-direction and m is the number of steps in height-direction.

As can be seen from figure 16, one step in width-direction is needed to get from the origin point to the stickers on the first column, three steps to get to the stickers on the second column, and five steps to get to the stickers on the third column. Same goes for the steps needed in height direction to get from the origin point to the stickers on first, second, and third row. Variable i stands for the row number and variable j for the column number of a sticker. In Python, index values start from 0 instead of 1, meaning that row and column number values 0, 1, and 2 stand for the first, second, and third row or column. Therefore, the number of steps in width-direction is

$$n = 2j + 1 \quad (13)$$

and the number of steps in height-direction is

$$m = 2i + 1 \quad (14)$$

The absolute y -coordinate of a given sticker center can be calculated as

$$y_{sticker} = y_0 + dy , \quad (15)$$

and the absolute x -coordinate as

$$x_{sticker} = x_0 + dx . \quad (16)$$

When substituting the results from equations 13 and 14 for n and m in equations 11 and 12, and then substituting the results from equations 11 and 12 for dx and dy in equations 15 and 16, the absolute coordinates can be presented as

$$y_{sticker} = y_0 + (2j + 1)dy_w + (2i + 1)dy_h \quad (17)$$

and

$$x_{sticker} = x_0 + (2j + 1)dx_w + (2i + 1)dx_h . \quad (18)$$

As shown in program 2, equations 7–10 are first used to calculate the values of transitions in directions x and y per one step in width- or height-direction. Then the step sizes are calculated using equations 5 and 6. After that, equations 17 and 18 are used inside the loop to calculate the absolute x - and y -values of the sticker centers.

```

# Computing the transitions on x and y axis when moving in the set of
# coordinates of the rotated cube.

if 60 <= rotation_deg <= 90:
    h, w = rectangle[1]

    # Checking that the contour is approximately square.
    if abs(w - h) > marginal:
        continue

    w_step = w // 6
    h_step = h // 6
    top_left = box[0]
    dx_per_w_step = sin(rotation_rad) * w_step
    dx_per_h_step = cos(rotation_rad) * h_step
    dy_per_w_step = -cos(rotation_rad) * w_step
    dy_per_h_step = sin(rotation_rad) * h_step

else: # if 0 < rotation_deg <= 30:
    w, h = rectangle[1]

    # Checking that the contour is approximately square.
    if abs(w - h) > marginal:
        continue

    w_step = w // 6
    h_step = h // 6
    top_left = box[1]
    dx_per_w_step = cos(rotation_rad) * w_step
    dx_per_h_step = -sin(rotation_rad) * h_step
    dy_per_w_step = sin(rotation_rad) * w_step
    dy_per_h_step = cos(rotation_rad) * h_step

x_0, y_0 = top_left

piece_centers = np.empty((3, 3), dtype="f,f")

# Drawing the contour of the detected face so user can see that
# the face is detected.
cv2.drawContours(img, [box], 0, (0, 0, 100), 3)

# Calculating and saving the locations of the piece centers.
for i in range(3):
    for j in range(3):
        piece_center_x = x_0 + (2*j + 1)*dx_per_w_step + (
            2*i + 1)*dx_per_h_step

        piece_center_y = y_0 + (2*j + 1)*dy_per_w_step + (
            2*i + 1)*dy_per_h_step

        piece_centers[i][j] = (piece_center_x, piece_center_y)

```

Program 2. *Calculating the locations of sticker centers (referred to as “piece centers” in the code).*

After the calculations represented above are completed and the sticker centers are located, the program continues to the next phase, where the color of each sticker is determined.

3.3.3 Detecting the colors of individual stickers

The third and the last phase of detecting the face being shown is to determine the color of each sticker after the outline of the face has been found and the locations of the stickers are calculated. Just like the whole cube was distinguished from the black background using a mask that does not let dark colors through, the different colors of the cube are being found using masks that filter away all but the wanted color from the image. In order to do this, six masks need to be created, one for each color of the cube. To create a mask that lets only the wanted color through, suitable limit values for the color need to be set. This was done by trial and error and using an online color picker [40].

It is crucial to change the color model from red-green-blue (RGB) to hue-saturation-value (HSV) in order to set limit values that work well in practice [41][42]. This is because changes in illumination affect all the three values of RGB-image making it difficult to detect colors correctly in different environments although the object being inspected stays the same. On the other hand, when using HSV color model, only saturation and value components of a color are significantly affected by the changes in lightning, but the hue component is relatively independent from these changes [43]. The hue describes the location of the color on the optical spectrum independent of the brightness or intensity of the color [43]. This makes it possible to distinguish different colors from each other correctly with pre-set limits without knowing the exact lighting features of the environment [43].

When all the six masks are created, the next step is to find all the contours of different shapes in the masked images. These contours outline the spots in the image where a given color can be found. In figure 17, three images are represented: the original image, the masked image where only the blue spots get through the filter, and the original image where all the blue spots are contoured using their locational data got from the masked image. In this case, the color blue would be called the key color of these contours.



Figure 17. Outlining all the blue spots on the image.

All the contours of the shapes colored any of the six colors of the cube are iterated through in a loop. Inside the loop, all the sticker center points are iterated through in another loop to check if they are inside the observed contour. If the center point of a given sticker is located inside the contour being inspected, the sticker is considered to be of a key color of that contour. If it happens to be the case, that the same sticker center is inside multiple contours with different key colors, the key color of the contour with the smallest area is used to determine the color of the sticker. One case where the comparison of the contour areas is crucial is represented in figure 18. When the center piece is surrounded by a ring of stickers that match with each other but do not match with the center piece itself, the center point of the piece is obviously inside both the contour of the surrounding ring and the contour of the piece itself. The surrounding contour that would lead to incorrect detection of a piece could also be some large shape on the background that encircles the whole cube. Thus, the areas of the contours need to be compared to prevent errors in detection.

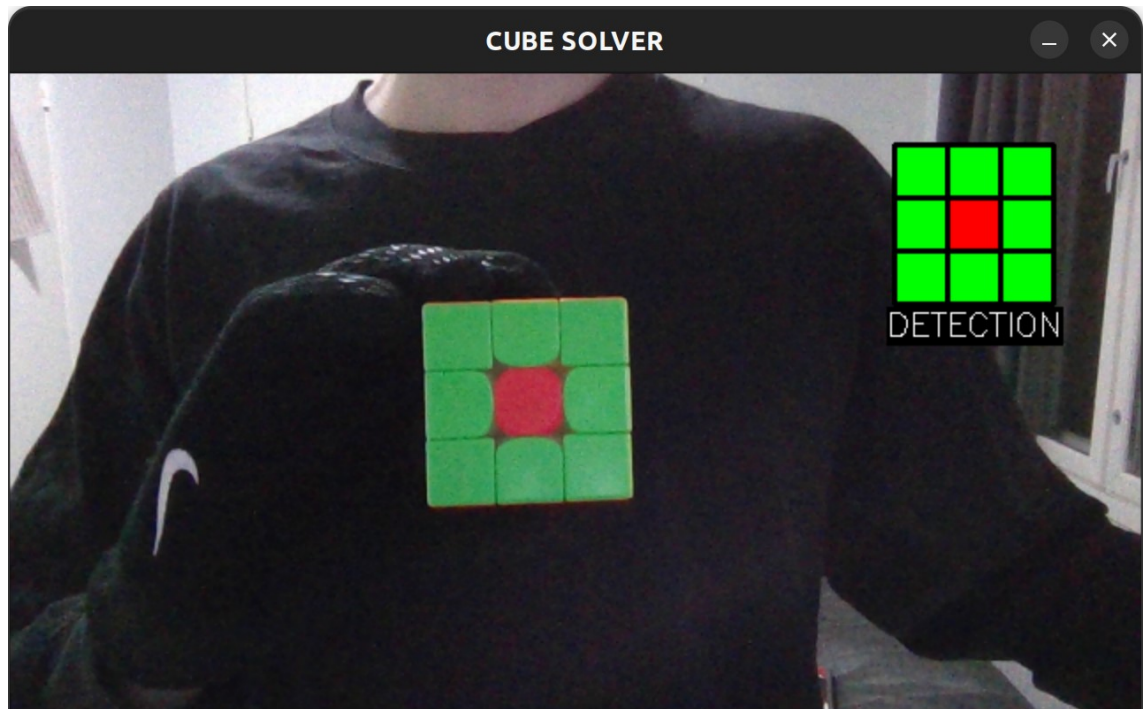


Figure 18. *Ring of green stickers surrounding a red center.*

As mentioned before, when all of the nine stickers have been detected, the user is asked to verify that the detection went right. The user can see the face, as it is detected, drawn on the top right corner of the window, as is shown in figure 7. If the face drawn on the screen and the face of the real cube do not match and user denies the verification, the detection process of the face starts again from the beginning. If the drawn face matches the face of the real cube as it should and the user verifies the detection, the program moves on to detect the next face. This goes on until the state of the whole cube is detected.

3.4 Solving the cube

After the state of the cube is correctly detected, the solution can be computed and then executed. As mentioned before, the solution is computed using Kociemba's algorithm, which can be found as a ready-made function from software library called Kociemba [5]. The function takes the current state of the cube as an input and returns a solution string where all the turns needed to solve the cube are written down. Kociemba's solve-function uses Singmaster notation, developed and named after American-British mathematician David Singmaster [44]. This style of notation, represented in figure 19, is almost identical to the one represented in figure 2. The difference is that when using Singmaster notation, the sides of the cube are named according to which directions they are pointing (up, left, front, right, back, or down) and when using the notation represented in figure 2, the sides

are named according to their final colors. The reason for the usage of figure 2 styled notation widely in the program is that the user is more likely to get confused when asked to show the front, right, or back side of the cube than when asked to show the blue, orange, or green side of the cube. However, when interacting with the function from Kociemba-library, the directions where each side of the cube is facing need to be determined. This is done by determining white face to be the one facing up and green face the one facing back (away from the user, towards the camera) and this notation is used through the whole solving phase.

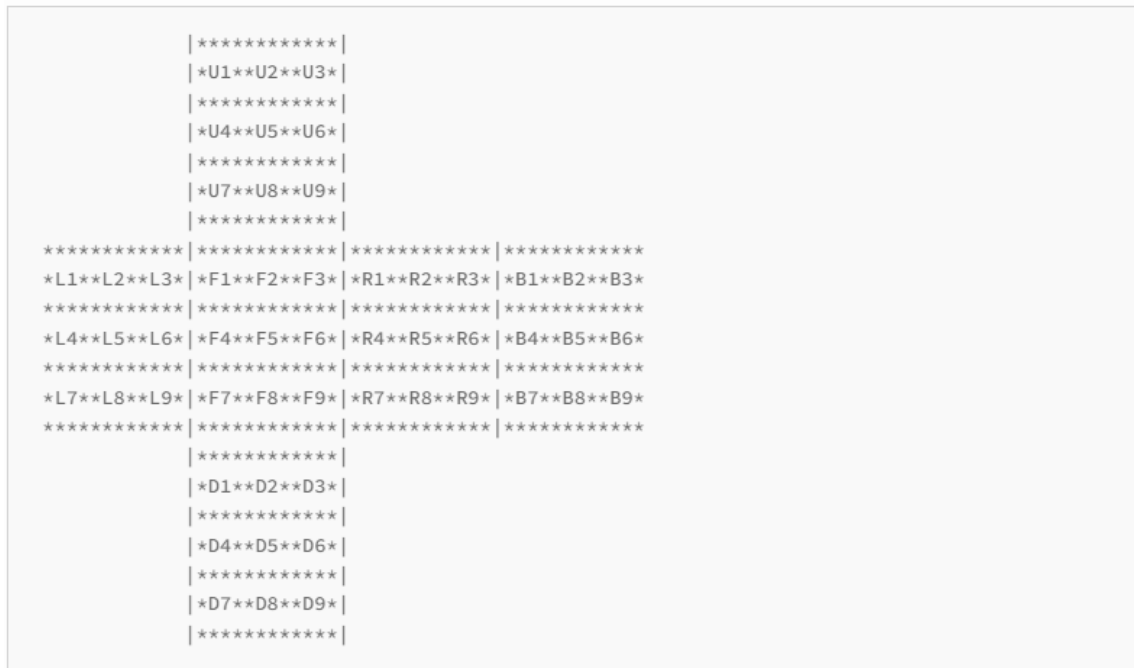


Figure 19. The sticker positions using Singmaster notation [5].

Kociemba's solve-function takes the state of the cube as a string where the colors of the stickers are represented as letters *U*, *L*, *F*, *R*, *B*, and *D* according to which side the sticker belongs to in a solved cube. The letters referring to the sticker colors need to be ordered according to their current positions on the cube. The order of the letters needs to match the following order of the positions: *U1...U9*, *R1...R9*, *F1...F9*, *D1...D9*, *L1...L9*, *B1...B9*. So, for example, the string representing the state of a solved cube would be "UUUUUUUUURRRRRRRRRRFFFFFFFFFDDDDDDDDLLLLLLLLLBBBBBBBBB". [5] Forming the input string for the solve-function starts by initializing an empty string and then iterating through attribute *state* of the *Cube* object, which structure is represented in program 1. On each iteration, the letter corresponding to the color name saved in the attribute *state* is added to the string. The corresponding letters for each color are *U* for white, *R* for orange, *F* for blue, *D* for yellow, *L* for red, and *B* for green.

Kociemba's solve-function computes a solution for the given state of the cube and returns a solution string which includes all the turns needed to solve the cube separated from each other with empty spaces. This string also uses Singmaster notation with letters *U*, *R*, *F*, *D*, *L*, and *B*, where each letter refers to quarter turn of the face of the same letter in clockwise direction. If there is a number 2 after the letter, for example *U2*, it means two quarter turns, in other words, a half turn. If there is a prime symbol after the letter, for example *U'*, it means a quarter turn in counterclockwise direction. [5] This style of notation is commonly used among cube enthusiasts.

After the solution string has been returned by Kociemba's solve-function, it is iterated through guiding the user to execute the solution turn by turn. The user is guided by the instruction text on the top of the window telling which face to show to the camera at any given moment and by the arrows drawn on the screen describing the desired rotations and turns of the cube. Examples of these instructions are represented in figures 6 and 8. The cube is constantly being inspected by the program during the solving phase using the same methods that were used during the detection phase. Each iteration starts by copying the face that should be shown to the camera from the attribute *state* of the *Cube*-object into a new variable. This represents the face before the turn is made. Then the turn method is called for the *Cube*-object updating all its faces correspondingly to the given turn. The face detected from the camera input is constantly compared to these two faces, the one preceding the turn and the one following it. Arrows guiding to make the turn stay on the screen as long as the face shown to the camera matches the pre-turn face. As soon as the face shown to the camera matches the post-turn face, the program moves on to the next turn on the solution string.

To make the guidance with the arrows as clear and unambiguous as possible, and to save the user from awkward hand positions, the *F*- and *B*-turns (turning the front or the back side of the cube) are always executed as *R*- or *L*-turns (turning the right or the left side of the cube) before which the cube is rotated. For example, if the green face is facing to the camera and it is also the one that should be turned, the user is first guided to rotate the whole cube so that the green face will be facing right and after that, the user is being guided to turn the right side of the cube, which now leads to a turn of the green face. This rotation will be undone when the next *R*- or *L*-turn appears on the solution string, so there will never be a situation where the user is asked to turn the side of the cube facing him-/herself or the camera.

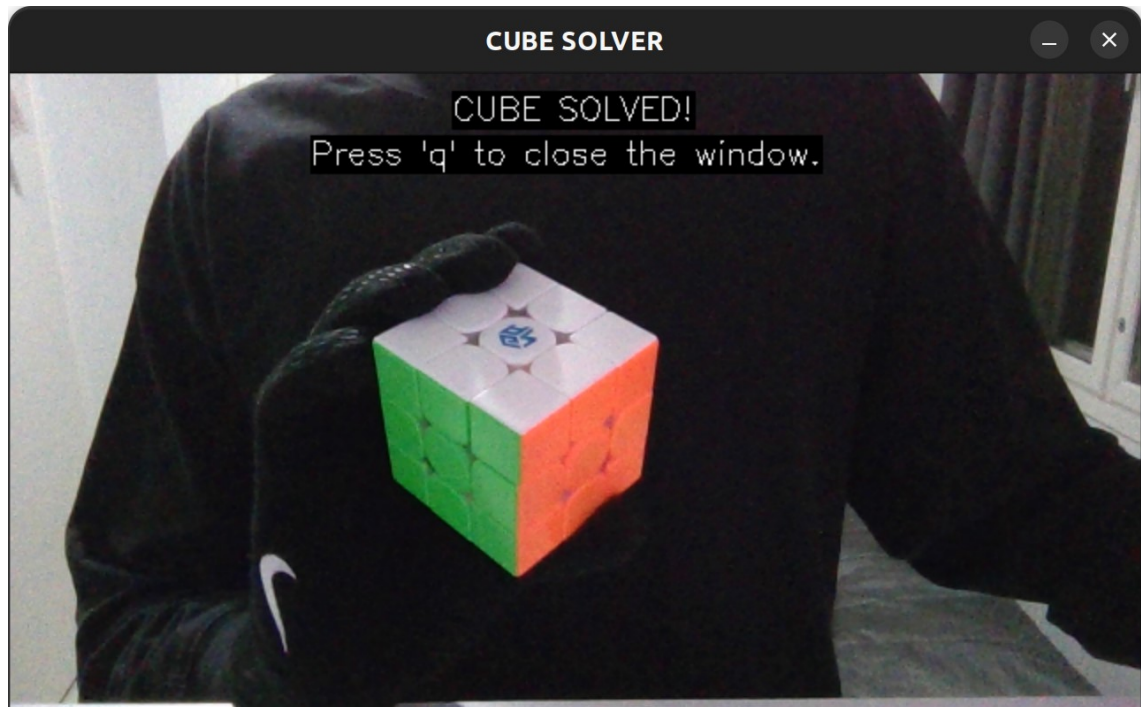


Figure 20. Achieving the goal.

The process described above goes on until there are no more turns left in the solution string and the cube is solved. Then the program announces that the cube is solved and guides the user to close the window. This view is represented in figure 20.

4. RESULTS AND POTENTIAL IMPROVEMENTS

The purpose of this thesis, as mentioned in introduction, was to create a program that can be used to solve the Rubik's Cube. Links to the complete Python code [45] and to a video of the program in action [46] can be found from the references.

The program created detects the state of a scrambled Rubik's Cube accurately when the limit values of the colors are set to be suitable for the environment. As long as the detection is successful, the program immediately finds a solution of around twenty moves which is either the shortest solution possible or close to it, and then guides the user to execute the solution with clear instructions constantly observing the actions of the user and the state of the cube. Thus, it can be said that the program meets the goals that were set during the planning phase.

Although the final implementation of the program works correctly most of the time, there are still some problematic situations where it does not perform successfully. First, the limit values of the six colors of the cube need to be set manually to correspond to the lighting conditions of the environment and if this is not done successfully, the detection of the faces will either fail or be slow and clumsy. Second, if the user does not follow the instructions shown on the screen and presents the faces by holding the cube incorrectly oriented (the desired face facing to the camera but incorrect face facing up) it will lead to flawed conclusions about the state of the cube and the rotational arrows drawn on the screen will be pointing to incorrect directions.

Based on the above, the program could be further improved by adding some kind of calibration into it, so that the suitable limit values for the colors would be automatically set according to lighting conditions. Early in the development process, this kind of calibration was part of the program, but manually setting the values outperformed the automatic calibration in detection accuracy constantly, so the calibration feature was left out from the final implementation. Another improvement would be to add more advanced detection method where multiple faces of the cube would be observed simultaneously enabling the program to conclude the orientation of the cube unambiguously from the image input by itself. This would allow the user to hold the cube in arbitrary positions during the detection and therefore allow more flexibility and freedom. The project could also be taken further by connecting the program to robotic hardware that manipulates the cube instead of the human operator enabling the system to be fully autonomous. This kind of a project could be done in the future as an extension of this thesis.

5. CONCLUSIONS

This thesis is about solving the Rubik's Cube with computer vision. The main purpose of the thesis was to develop a program that is able to detect the state of a Rubik's Cube side by side, compute a solution for it, and guide a human user to execute the solution. Another goal of the thesis, besides the programming project, was to examine computer vision as a discipline and to get familiar with the tools and practices commonly used in the field.

Multiple tools and platforms to use when working on a computer vision project are presented in subchapter 2.2.3. From the wide variety of options, Python was chosen to be the programming language, OpenCV library was chosen to be used in computer vision tasks, and Kociemba's algorithm was chosen to be used for computing solutions for given states of the cube. These three seem to be highly popular choices to use in this kind of task compared to their alternatives among the software developer community.

The program is implemented using only traditional computer vision techniques that are based on detecting colors and shapes from an image and following hard-coded rules. More advanced state-of-the-art methods utilizing deep learning are also discussed in subchapter 2.2.2, but using those is not necessary for this kind of task. However, some features of the program could indeed be improved by using this kind of advanced techniques.

The final version of the program seems to meet the goals that were set during the planning. It is able to solve a Rubik's Cube from any given starting position. There are still some features in the program that could be further improved, which are more precisely discussed in chapter 4. The two most important improvements would be an automatic calibration of the colors' limit values and multi-facial cube detection which would allow the program to recognize the orientation of the cube without restricting the user to hold the cube only in predetermined positions.

The program created could also have some interesting future applications. It could be connected to robotic hardware, for example simple claws, that would operate the cube instead of a human user. Building this kind of fully autonomous Rubik's Cube solver is a potential future project to continue the work from this thesis.

REFERENCES

- [1] About Rubik's Cube, Rubik's, 2024, Available (Accessed 15.1.2024): <https://rubiks.com/en-US/>
- [2] M. O'Leary, Featured video: Solving a Rubik's Cube in record time, MIT News, 2018, Available (Accessed Jan 15, 2024): <https://news.mit.edu/2018/featured-video-solving-rubiks-cube-record-time-0316>
- [3] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, L. Zhang, Solving Rubik's Cube with a robot hand, OpenAI, 2019, Available (Accessed Jan 15, 2024): <https://openai.com/research/solving-rubiks-cube>
- [4] Gan Robot – Intelligent Cubing Pioneer, Gan Cube, Available (Accessed Jan 15, 2024): <https://www.gancube.com/gan-robot/>
- [5] M. Tsoy, kociemba 1.2.1, PyPi, 2019, Available (Accessed Jan 15, 2024): <https://pypi.org/project/kociemba/>
- [6] Cube Facts, Rubik's, 2018, Available (Accessed Jan 15, 2024): <https://web.archive.org/web/20180309125149/https://www.rubiks.com/about/cube-facts>
- [7] T. Rokicki, H. Kociemba, M. Davidson, J. Dethridge, The Diameter of the Rubik's Cube Group is Twenty, Society for Industrial and Applied Mathematics, 2013, pp. 1082–1083. Available (Accessed Jan 15, 2024): <http://kociemba.org/math/papers/rubik20.pdf>
- [8] Computer Vision: What it is and why it matters, SAS Analytics Insights, Available (Accessed Jan 15, 2024): https://www.sas.com/en_id/insights/analytics/computer-vision.html
- [9] Z. Zou, K. Chen, Z. Shi, Y. Guo and J. Ye, Object Detection in 20 Years: A Survey, Proceedings of the IEEE, Vol. 111, No. 3, Mar 2023, pp. 257-276, DOI: [10.1109/JPROC.2023.3238524](https://doi.org/10.1109/JPROC.2023.3238524)

- [10] J. Enoch, L. McDonald, L. Jones, P.R. Jones, D.P. Crabb, Evaluating Whether Sight Is the Most Valued Sense, *JAMA Ophthalmol*, Vol. 137, No. 11, Nov 2019, pp. 1317–1320, DOI: [10.1001/2Fjamaophthalmol.2019.3537](https://doi.org/10.1001/2Fjamaophthalmol.2019.3537)
- [11] F. Hutmacher, Why Is There So Much More Research on Vision Than on Any Other Sensory Modality? *Frontiers in psychology*, Vol. 10, Oct 2019, pp. 2246, DOI: [10.1001/2Fjamaophthalmol.2019.3537](https://doi.org/10.1001/2Fjamaophthalmol.2019.3537)
- [12] Autopilot and Full Self-Driving Capability, Tesla – Support, Available (Accessed Jan 15, 2024): https://www.tesla.com/en_eu/support/autopilot
- [13] Automated Straddle Carrier Application, Kalmar Global, Available (Accessed Jan 15, 2024): <https://www.kalmarglobal.com/automation/kalmarone/autostrad-application/>
- [14] Sandvik Mining Automation, Mining Technology, Available (Accessed Jan 15, 2024): <https://www.mining-technology.com/products/sandvik-mining-automation/>
- [15] About Face ID advanced technology, Apple – Support, Available (Accessed Jan 15, 2024): <https://support.apple.com/en-us/102381>
- [16] New CyberLink Report Finds Over 131 Million Americans Use Facial Recognition Daily and Nearly Half of Them to Access Three Applications or More Each Day, *Business Wire*, Available (Accessed Jan 15, 2024): <https://www.business-wire.com/news/home/20221123005021/en/New-CyberLink-Report-Finds-Over-131-Million-Americans-Use-Facial-Recognition-Daily-and-Nearly-Half-of-Them-to-Access-Three-Applications-or-More-Each-Day>
- [17] S.M. McKinney, M. Sieniek, V. Godbole, J. Godwin, N. Antropova, H. Ashrafian, T. Back, M. Chesus, G.S. Corrado, A. Darzi, M. Etemadi, F. Garcia-Vicente, F.J. Gilbert, M. Halling-Brown, D. Hassabis, S. Jansen, A. Karthikesalingam, C.J. Kelly, D. King, J.R. Ledsam, D. Melnick, H. Mostofi, L. Peng, J.J. Reicher, B. Romera-Paredes, R. Sidebottom, M. Suleyman, D. Tse, K.C. Young, J. De Fauw, S. Shetty. International evaluation of an AI system for breast cancer screening, *Nature*, Vol. 577, 2020, pp. 89–94, DOI: [10.1038/s41586-019-1799-6](https://doi.org/10.1038/s41586-019-1799-6)
- [18] D. Kitaguchi, N. Takeshita, H. Hasegawa, M. Ito, Artificial intelligence-based computer vision in surgery: Recent advances and future perspectives, *Annals of Gastroenterological Surgery*, Vol. 6, 2022; pp. 29–36, DOI: [10.1002/ags3.12513](https://doi.org/10.1002/ags3.12513)
- [19] P. Mascagni, D. Alapatt, L. Sestini, M.S. Altieri, A. Madani, Y. Watanabe, A. Alseidi, J.A. Redan, S. Alfieri, G. Costamagna, I. Boškoski, N. Padoy, D.A. Hashimoto, Computer vision in surgery: from potential to clinical value, *npj Digital Medicine*, Vol. 5, No. 163, Oct 2022, DOI: [10.1038/s41746-022-00707-5](https://doi.org/10.1038/s41746-022-00707-5)
- [20] T.M. Ward, P. Mascagni, Y. Ban, G. Rosman, N. Padoy, O. Meireles, D.A. Hashimoto, Computer vision in surgery, *Surgery*, Vol. 169, No. 5, pp. 1253–1256, DOI: [10.1016/j.surg.2020.10.039](https://doi.org/10.1016/j.surg.2020.10.039)

- [21] E. David, Deep Learning for Dummies, John Wiley & Sons, Inc, 2018, Chapter 1, Available: <https://learning.oreilly.com/library/view/deep-learning-for/9781119543046/c01.xhtml#h2-4>
- [22] S.J. Denxler, L.E. van Dyck, W.R. Gruber, R. Kwitt, Comparing Object Recognition in Humans and Deep Convolutional Neural Networks—An Eye Tracking Study, Frontiers in Neuroscience, Vol. 15, 2021, DOI: [10.3389/fnins.2021.750639](https://doi.org/10.3389/fnins.2021.750639)
- [23] Y. Noema, Top 3 Programming Languages For Implementing a Computer Vision System, Medium, Dec 2021, Available (Accessed Jan 15, 2024): <https://medium.com/imagescv/my-top-3-programming-languages-for-implementing-a-computer-vision-system-2aa0a3ad0a2a>
- [24] Mathieu, Top AI Programming Languages for 2023, Bocasay, Jul 2023, Available (Accessed Jan 15, 2024): <https://www.bocasay.com/top-ai-programming-languages-2023/#>
- [25] P. Ingle, Top Computer Vision Tools/Platforms in 2023, Marktechpost, Jul 2023, , Available (Accessed Jan 15, 2024): <https://www.marktechpost.com/2023/07/17/top-computer-vision-tools-platforms-in-2022/>
- [26] G. Boesch, The 12 Most Popular Computer Vision Tools in 2024, Viso.ai, 2023, Available (Accessed Jan 15, 2024): <https://viso.ai/computer-vision/the-most-popular-computer-vision-tools/>
- [27] J. Terra, Keras vs Tensorflow vs Pytorch: Key Differences Among Deep Learning, Simplilearn, Aug 2023, Available (Accessed Jan 15, 2024): https://www.simplilearn.com/keras-vs-tensorflow-vs-pytorch-article#what_is_pytorch
- [28] L. Ruchzyński, ML Frameworks Compared: Scikit-Learn, Tensorflow, PyTorch and More, Netguru, Dec 2022, Available (Accessed Jan 15, 2024): <https://www.netguru.com/blog/top-machine-learning-frameworks-compared>
- [29] R. Kanjee, 10 Epic Tools for Computer Vision Developers in 2023, Medium, Jun 2023, Available (Accessed Jan 15, 2024): <https://augmentedstartups.medium.com/10-epic-tools-for-computer-vision-developers-in-2023-a72343ffdd60>
- [30] J. Cleveland, Top 10 Computer Vision Tools in 2023, Jonas Cleveland, Available (Accessed Jan 15, 2024): <https://jonascleveland.com/computer-vision-tools/>
- [31] Top 10 Computer Vision Tools and Computer Vision Use Cases, Websmith Solution, 2022, Available (Accessed Jan 15, 2024): <https://websmithsolution.com/computer-vision-tools/>
- [32] Top 16 computer vision libraries, SuperAnnotate, May 2023, Available (Accessed Jan 15, 2024): <https://www.superannotate.com/blog/computer-vision-libraries>
- [33] About, OpenCV, Available (Accessed Jan 15, 2024): <https://opencv.org/about/>

- [34] Fastest robot to solve a puzzle cube, Guinness World Records, Available (Accessed Jan 15, 2024): <https://www.guinnessworldrecords.com/world-records/fastest-robot-to-solve-a-rubiks-cube>
- [35] J. Di Carlo, B. Katz, MIT Robot Breaks Rubik's Cube Record, Mar 2018, Available: <https://www.youtube.com/watch?v=OZu9gjQJUQs>
- [36] B. Katz, The Rubik's Contraption, Build Its in Progress, Mar 2018, Available (Accessed Jan 15, 2024): <https://build-its-inprogress.blogspot.com/2018/03/the-rubiks-contraption.html>
- [37] J. Di Carlo, Rubik's Solver Software, The Cactus Zone, Mar 2018, Available (Accessed Jan 15, 2024): <https://cactus-zone.blogspot.com/2018/03/rubiks-solver-software.html>
- [38] findContours(), OpenCV – Docs, Available (Accessed Jan 15, 2024): https://docs.opencv.org/4.x/d3/dc0/group_imgproc_shape.html#gadf1ad6a0b82947fa1fe3c3d497f260e0
- [39] minAreaRect(), OpenCV – Docs, Available (Accessed Jan 15, 2024): https://docs.opencv.org/4.x/d3/dc0/group_imgproc_shape.html#ga3d476a3417130ae5154aea421ca7ead9
- [40] HTML Color Picker, RapidTables, Available (Accessed Jan 15, 2024): <https://www.rapidtables.com/web/color/color-picker.html>
- [41] M. Nursabillilah, N. Rashid, M. Yasir, Performance Comparison between RGB and HSV Color Segmentations for Road Signs Detection, Applied Mechanics and Materials, Vol. 393, Sep 2013, pp. 550–555. DOI: [10.4028/www.scientific.net/amm.393.550](https://doi.org/10.4028/www.scientific.net/amm.393.550)
- [42] A. Ajmal, C. Hollitt, M. Freat, H. Al-Sahaf, A Comparison of RGB and HSV Colour Spaces for Visual Attention Models, International Conference on Image and Vision Computing New Zealand, Auckland, New Zealand, 2018, pp. 1–6. DOI: [10.1109/IVCNZ.2018.8634752](https://doi.org/10.1109/IVCNZ.2018.8634752)
- [43] HUE, VALUE, SATURATION, Learn, Jun 2017, Available (Accessed Jan 15, 2024): <https://web.archive.org/web/20170630065346/http://learn.leigh-cotnoir.com/artspeak/elements-color/hue-value-saturation/>
- [44] D. Singmaster, Notes on Rubik's Magic Cube, Enslow Publishers, 1981, pp. 3–4, Available: <https://maths-people.anu.edu.au/~burkej/cube/singmaster.pdf>
- [45] L. Jokinen, RubiksCubeSolver, GitHub, Jan 2024, Available (Accessed Jan 17, 2024): <https://github.com/JokinenL/RubiksCubeSolver>
- [46] L. Jokinen, Solving the Rubik's Cube with OpenCV, YouTube, Jan 2024, Available (Accessed Jan 17, 2024): <https://youtu.be/SUVpxA4IFMl>